INE Skill Check Lab Report

Completing Skill Check Labs – Web Reconnaissance

Target: **http://target.ine.local**
Assessment Type: **Web Reconnaissance / Information Disclosure**
Tools Used: **Firefox, Curl, HTTrack, Nikto, Dirb**

Flag	Value
Flag 1	FLAG1{4054316878a14367a23a81b68d7b8289}
Flag 2	FLAG2{b39ea91089694a178655307ba6cfd596}
Flag 3	FLAG3{25de5f11aa9949e8902d9129f1b2dd37}
Flag 4	FLAG4{d03b9ec5db564d4abe52bff1c75365c2}
Flag 5	FLAG5{246fd9f770984cf8b9182bf2cad9feac}

Executive Summary

This report documents the complete solution for the INE Skill Check Lab focused on web reconnaissance. The target was identified as a WordPress site exposing multiple information-disclosure weaknesses, including a publicly accessible robots.txt file, generator metadata in the source code, directory browsing on uploads, an exposed backup configuration file, and a mirrored XML-RPC related file that revealed the final flag.

Methodology

The assessment was performed in a structured sequence: initial browsing and source inspection, HTTP enumeration, directory and file discovery, targeted validation of exposed paths, and offline analysis of mirrored content. Each flag below includes the discovery method, the evidence, and the captured value.

Flag 1 – robots.txt Disclosure

The first flag was obtained by visiting the robots.txt file. This file informs search engines which paths should be indexed or avoided, and it often leaks useful administrative or hidden paths.

URL: http://target.ine.local/robots.txt

FLAG1{4054316878a14367a23a81b68d7b8289}

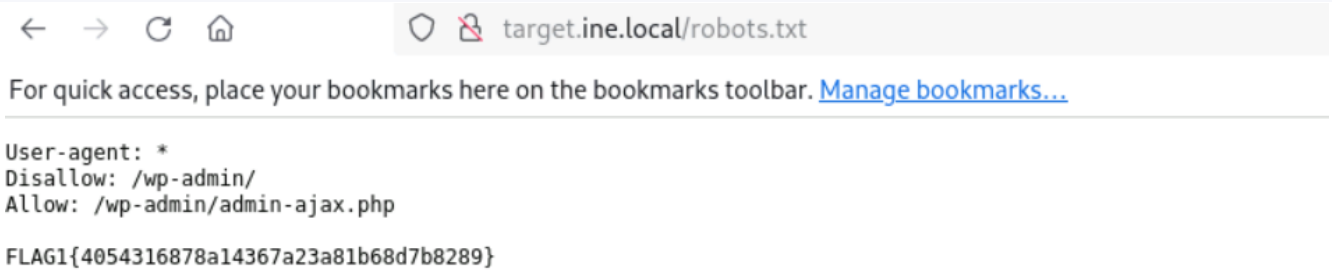

Figure – Evidence for Flag 1 – robots.txt Disclosure

Flag 2 – Website Identification and Version Enumeration

The second flag was identified from the page source. The HTML generator metadata revealed that the target application is WordPress and exposed its exact version. This confirms both the underlying technology and the version in use.

Evidence: meta generator tag in the page source showed WordPress 6.5.3 along with the embedded flag.

FLAG2{b39ea91089694a178655307ba6cfd596}

Figure – Evidence for Flag 2 – Website Identification and Version Enumeration

Flag 3 – Directory Browsing in Uploads

The third flag was found after identifying that directory listing was enabled for the uploads directory. This is a common information-disclosure issue because it allows unauthenticated users to enumerate uploaded content directly from the browser.

Discovered path: /wp-content/uploads/

FLAG3{25de5f11aa9949e8902d9129f1b2dd37}

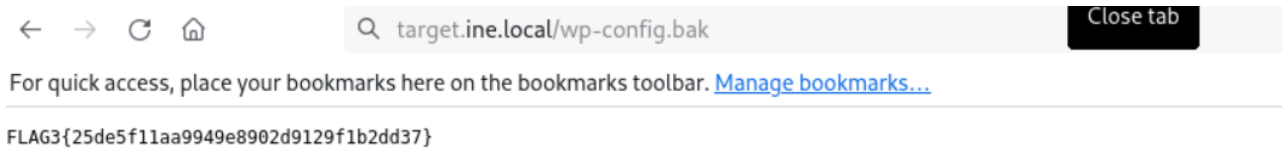

Figure – Evidence for Flag 3 – Directory Browsing in Uploads

Flag 4 – Exposed Backup File in Webroot

The fourth flag was extracted from an exposed backup copy of the WordPress configuration file. Backup files left in the webroot are dangerous because they may reveal secrets, salts, database configuration, or internal application details.

Discovered file: /wp-config.bak

```
FLAG4{d03b9ec5db564d4abe52bff1c75365c2}
```

```
* This will force all users to have to log in again.
*
* @since 2.6.0
*/
define('AUTH_KEY',         '}Mq^#|v{n0fQ6Vn[tr 6e4glzi:OVs/9(IQ .7f^dp3ym4,th-O$Qx.]|2+(t(sE');
define('SECURE_AUTH_KEY',  'S_LKQ#*}p*U}kdX[GNNVM2*0YISNQ&zrFl jEUNq5T}0Zg|,sO|yB68^|N*1nS-p');
define('LOGGED_IN_KEY',    '`tz-Uz9Ixka,5z0J BD0l/zfU|r2|;9BGL5l~A1RQtZMwh=JftaU$2)$FI%v};|E');
define('NONCE_KEY',        '▷ZN961k>aHWJ*R8#&x+rR>3g|<[:G 8B+rqPH WrWet1SC60+ LL/S+=[G-&g7)');
define('AUTH_SALT',        '+<2l=;osCL(L)zV[=uvr[}2^j-16(gFq18V<m|fP<R{7DV`^=O&bb3fxY+Jf├-;C');
define('SECURE_AUTH_SALT', 'HG&/Q/ceR-$;?jCL}<cL4@LKzDjv,M=K-gR<]iHiAqcHQO+rXcWn/jMt0#K,uWq%');
define('LOGGED_IN_SALT',   'REsFv+OsL*qd=yV<oPaAXeYj@f)A[/Wm5-?|_4d::(;dXcps`rgJf]t4B0Q3)RcH');
define('NONCE_SALT',       ' Q.:O=pFDTA-lNBe%kjJu(mp7$cQrF|IZ _hOWDA&Q18w6CL(<{+1$a-ZJ~<(!_;');


/** FLAG4{d03b9ec5db564d4abe52bff1c75365c2} */


/**#@-*/

/**
* WordPress database table prefix.
*
* You can have multiple installations in one database if you give each
* a unique prefix. Only numbers, letters, and underscores please!
*/
$table_prefix = 'wp_';

/**
* For developers: WordPress debugging mode.
*
* Change this to true to enable the display of notices during development.
* It is strongly recommended that plugin and theme developers use WP_DEBUG
* in their development environments.
```

Figure – Evidence for Flag 4 – Exposed Backup File in Webroot

Flag 5 – Mirrored File Analysis with HTTrack

The final flag was recovered after mirroring the target website and reviewing the downloaded files. The mirrored content included a file named xmlrpc0db0.php, and reading that file revealed the hidden flag value inside the XML content.

Mirrored file: xmlrpc0db0.php

FLAG5{246fd9f770984cf8b9182bf2cad9feac}

```
┌──(root㉿INE)-[~/site/target.ine.local]
└─# cat xmlrpc0db0.php
<?xml version="1.0" encoding="UTF-8"?><rsd version="1.0" xmlns="http://archipelago.phrasewise.com/rsd">
        <service>
                <engineName>WordPress</engineName>
                <engineLink>https://wordpress.org/</engineLink>
                <homePageLink>http://target.ine.local</homePageLink>
                <apis>
                        <api name="WordPress" blogID="1" preferred="true" apiLink="http://target.ine.local/xmlrpc.php" />
                        <api name="Movable Type" blogID="1" preferred="false" apiLink="http://target.ine.local/xmlrpc.php" />
                        <api name="MetaWeblog" blogID="1" preferred="false" apiLink="http://target.ine.local/xmlrpc.php" />
                        <api name="Blogger" blogID="1" preferred="false" apiLink="http://target.ine.local/xmlrpc.php" />
                        <api name="FLAG5{246fd9f770984cf8b9182bf2cad9feac}" blogID="1" preferred="false" apiLink="http://target.ine.local/xmlrpc.php" />
                                <api name="WP-API" blogID="1" preferred="false" apiLink="http://target.ine.local/index.php/wp-json/" />
                        </apis>
        </service>
</rsd>
```

Figure – Evidence for Flag 5 – Mirrored File Analysis with HTTrack

Conclusion

The lab demonstrates how small reconnaissance findings can quickly chain into meaningful disclosure. A simple robots.txt file revealed sensitive hints, source inspection exposed technology and versioning, directory browsing disclosed file storage locations, an overlooked backup file leaked internal configuration, and offline mirroring uncovered additional hidden content. Together, these findings illustrate why routine web hardening, backup hygiene, and access control reviews are critical for WordPress deployments.